Singleton Pattern Case 1: Inventory Manager

Identified Problem

In the E-OMS system, multiple components interact with product inventory, including product catalog management and order processing. Each of these components must read and update the same stock information. If multiple inventory manager objects were created, different parts of the system could operate on different copies of the inventory data, leading to inconsistent stock values and incorrect order processing.

Why the Singleton Pattern Was Selected

The Singleton pattern ensures that a class has **only one instance** and provides a **global access point** to that instance. This makes it suitable for managing shared resources that must remain consistent across the entire application.

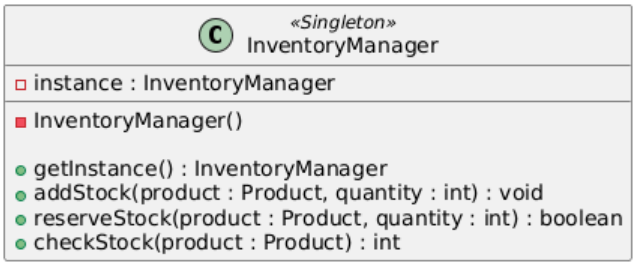
How the Pattern Solves the Problem

By implementing the Inventory Manager as a Singleton, the system guarantees that:

- Only one inventory manager instance exists during the application's execution.
- All controllers and system components access the same inventory state.
- Inventory updates remain consistent across the system.

This design ensures centralized control of inventory operations and prevents inconsistencies caused by multiple inventory objects.

UML:



Singleton Pattern Case 2: Logger

Identified Problem

In the E-OMS system, several components such as controllers and services need to record system events, informational messages, and errors during execution. If each component created its own logging object, the system would produce inconsistent logging behavior and unnecessary duplication of logging resources.

This would make system monitoring and debugging more difficult because log messages would originate from multiple independent logger instances instead of a single centralized source.

Why the Singleton Pattern Was Selected

The Singleton pattern ensures that a class has **only one instance** and provides a **global point of access** to that instance.

A logging service is a typical example where a single shared object is required to coordinate system logging across different modules.

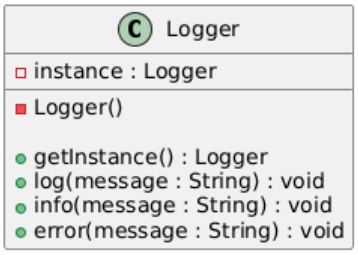
How the Pattern Solves the Problem

The Logger class is implemented as a Singleton so that:

- Only one logger instance exists during the application's runtime.
- All system components use the same logging service.
- Logging behavior remains consistent across the system.

This design centralizes system logging and ensures that all log messages are produced through a single controlled instance.

UML:



Abstract Factory Pattern Case 1: User Interface Configuration

Identified Problem

The system supports multiple user roles, specifically **Customer** and **Admin**, where each role requires a different set of user interface components such as menus, dashboards, and forms.
Each role must use a consistent group of UI components that belong together.

If the application directly creates these components using conditional logic (e.g., `if/else` statements) or concrete class references, the code becomes tightly coupled to specific implementations. This makes the system harder to maintain and extend, especially if new roles or UI configurations need to be introduced.

Why the Abstract Factory Pattern Was Selected

The Abstract Factory pattern provides an interface for creating **families of related objects without specifying their concrete classes**.

This pattern is appropriate for this case because the system needs to create groups of related UI components that must always be used together for a specific user role.

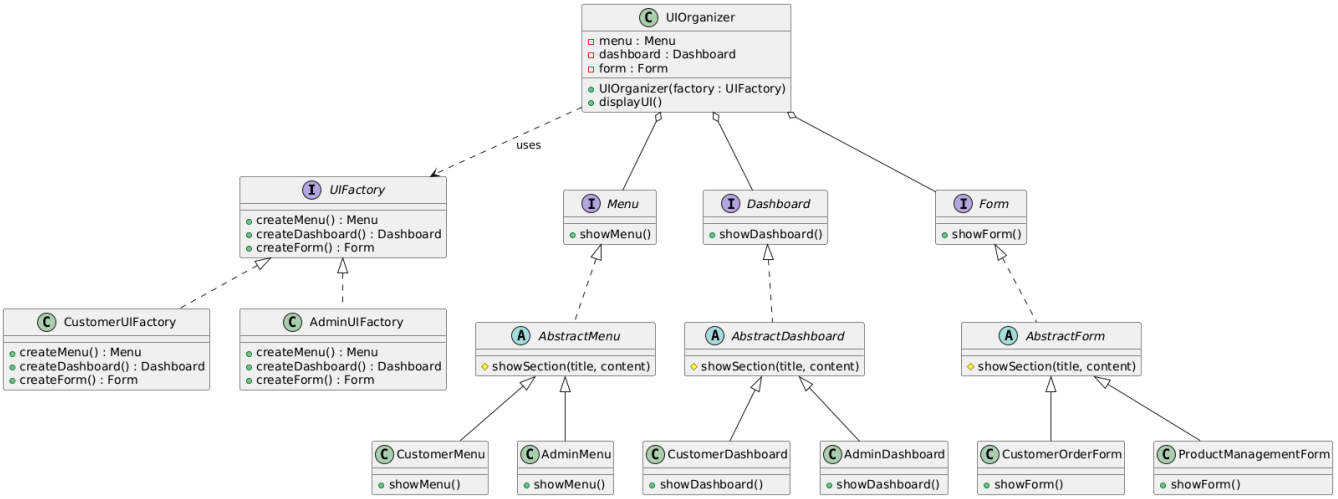
How the Pattern Solves the Problem

By applying the Abstract Factory pattern:

- Each user role has its own concrete UI factory that creates the appropriate UI components.
- The application interacts only with abstract interfaces for the UI components and the factory.
- The system can switch between different UI configurations without modifying the client code.

This design ensures that the correct set of UI components is created for each user role while keeping the system flexible and easier to extend when new UI configurations or roles are introduced.

UML:



Abstract Factory Pattern Case 2: Product Type Processing

Identified Problem

The order processing functionality must support different types of products such as **physical products, digital products, and services**.
Each product type follows different rules for operations such as inventory handling and invoice calculation.

If the system directly implements these differences using conditional logic (e.g., `if/else` statements) inside the business logic, the order processing classes become tightly coupled to specific product types. This results in complex and difficult-to-maintain code. Additionally, introducing a new product type would require modifying existing classes, which violates good software design practices.

Why the Abstract Factory Pattern Was Selected

The Abstract Factory pattern provides an interface for creating **families of related objects without specifying their concrete classes**.

This pattern is appropriate because each product type requires a consistent set of related behaviors (such as stock handling and invoice calculation) that must work together.

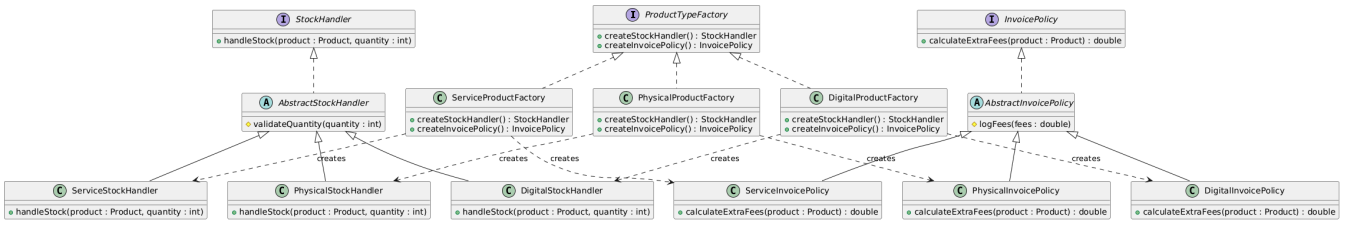
How the Pattern Solves the Problem

By applying the Abstract Factory pattern:

- Each product type has its own concrete factory responsible for creating the related components required for processing that product.
- The order processing logic interacts only with abstract interfaces rather than concrete classes.
- New product types can be introduced by adding new factories and corresponding classes without modifying existing business logic.

This approach improves maintainability and allows the system to support new product categories while keeping the order processing code clean and extensible.

UML:



Factory Method Pattern Case: Payment Processing

Identified Problem

The system must support multiple payment methods such as **Cash on Delivery, Credit Card, and PayPal**, where each payment type has its own processing logic.
If the application directly creates payment objects inside the order-processing logic using conditional statements (e.g., `if/else` or `switch`), the code becomes tightly coupled to specific payment implementations. This makes the system harder to maintain and extend. For example, adding a new payment method would require modifying existing business logic.

Why the Factory Method Pattern Was Selected

The Factory Method pattern defines an interface for creating objects while allowing subclasses to decide which concrete class to instantiate.

This approach is suitable because the system needs to create different payment objects depending on the selected payment type while keeping the order-processing logic independent from the specific implementations.

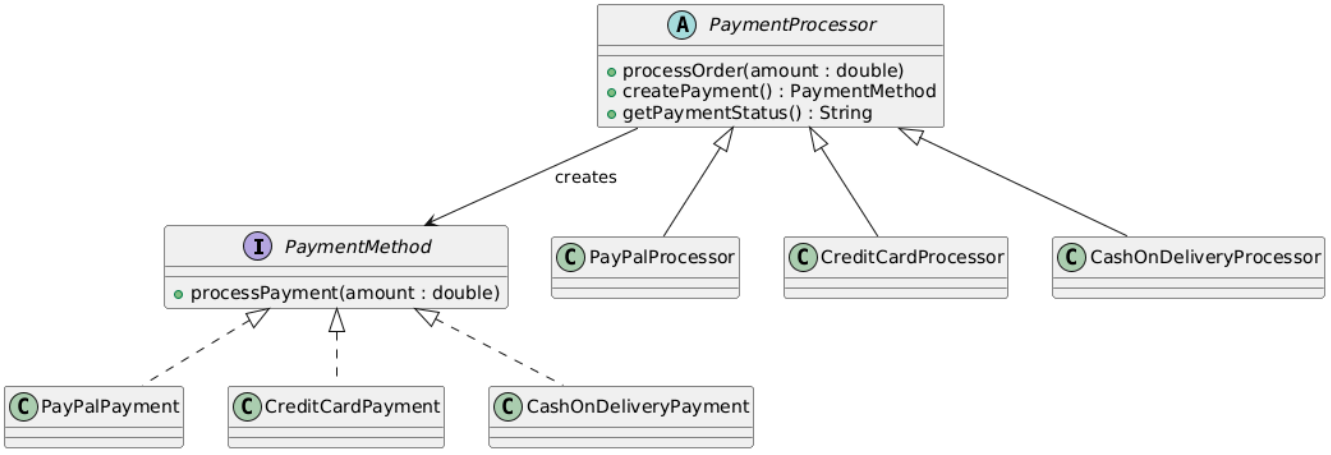
How the Pattern Solves the Problem

By applying the Factory Method pattern:

- Payment creation is delegated to specialized processor classes.
- The order-processing logic works with an abstract payment interface instead of concrete classes.
- New payment methods can be added by introducing new subclasses without modifying existing code.

This design separates the creation of payment objects from their usage, making the payment processing module more flexible and easier to extend.

UML:



GitHub repository

https://github.com/Kiro-create/NexOrder